Informe de operación — Práctica 10

Carnet: **202200075**. Prueba del 17 de octubre de 2026. Horas expresadas en UTC (Guatemala: UTC−6). Entorno: GKE `sa-p9`, tres nodos, servicios en `sa-p9` y stack de observabilidad en `observabilidad`.

1. SLO

Disponibilidad: **99 % de peticiones sin HTTP 5xx**. Latencia: **95 % de peticiones completadas en 500 ms o menos**. Ambos usan una ventana de **24 horas** y miden el gateway, excluyendo sondas y métricas. Las reglas calculan indicadores, presupuestos y consumo acelerado en ventanas de 1 y 5 minutos. La instalación todavía no dispone de 24 horas completas de historial; los resultados representan el período observado. El fallo controlado consume el presupuesto de latencia. El 99 % reconoce la disponibilidad limitada de un clúster zonal académico. El 95 % bajo 500 ms exige una interacción ágil y reserva un 5 % para operaciones ocasionalmente costosas. En la captura final de las 13:56 UTC, el SLI de disponibilidad fue **100 %** y el de latencia **93.90 %**: los ensayos consumieron **121.96 %** del presupuesto de latencia. La ventana conserva ese efecto después de recuperar el servicio.

2. Fallo inducido

Se publicó `FAULT_DELAY_MS=2500` únicamente en la candidata de catálogo mediante GitOps, revisión `b014533`. El primer registro del fallo apareció a las **13:36:06.207 UTC**. La candidata recibió el 50 % del tráfico durante cuatro minutos antes del análisis. La demora afecta GraphQL y conserva las sondas y las respuestas HTTP 200; se distingue del fallo de readiness de P8. El AnalysisRun `catalogo-b598b97bb-5-2` midió **26.4 peticiones** en el minuto de evaluación, error HTTP **0** y p95 de **2.995 y 3.900 segundos**, superando el límite de 0.5 s.

3. Tiempo de detección

La primera señal de alerta, **pending**, comenzó a las **13:37:18.993 UTC**, unos **73 segundos (1.21 min)** después del primer evento del fallo. `P10LatencyBudgetBurn` se observó en estado **firing a las 13:37:37.273 UTC**: aproximadamente **91 segundos (1.52 min)** después del primer evento. Se consultó la API cada tres segundos; ese intervalo limita la precisión del instante observado. Se comprobó que la alerta estaba inactiva antes de iniciar la prueba.

4. Recorrido del síntoma a la causa

Alerta de presupuesto → Tempo, traza `5280d95c9c5c55c5025a8f9935cba757`, duración **2629 ms** → Loki, mismo `trace_id` y evento `catalogo_dependency_latency`. El registro identifica `delay_ms=2500` y la causa `latencia_inducida_dependencia_catalogo`.

La medición automatizada por API obtuvo la correlación en **1.644 s** y terminó todas las consultas de diagnóstico en **4.999 s**, por debajo del máximo de 180 s. Se conservan las marcas originales y el cálculo separado de ese tiempo completo. La navegación visual se muestra en el video. Una operación normal adicional conserva una misma traza con spans de gateway, solicitudes y catálogo.

5. Tiempo de recuperación

Argo Rollouts abortó automáticamente la candidata a las **13:40:20 UTC**, unos **254 segundos (4.23 min) después del primer evento del fallo**. El análisis falló por p95; las comprobaciones de tráfico y errores pasaron. Después de la reversión se verificaron únicamente endpoints de catálogo estable y una operación de tráfico de **183 ms**; esa observación ocurrió aproximadamente

295 segundos (4.91 min) después del inicio. La revisión GitOps `c874b98` restauró `FAULT_DELAY_MS=0`, y la raíz y plataforma volvieron a Healthy.

La continuidad de P9 se comprobó comparando los mismos identificadores, contenidos y fechas, recuperando un usuario cifrado existente y verificando secretos y respaldos Completed. No se destruyeron bases, clúster ni volúmenes.

6. Alerta faltante o ruidosa

La alerta de latencia se activó y permitió localizar la regresión. La alerta de disponibilidad permaneció inactiva porque el fallo conservó HTTP 200; ese comportamiento es el esperado. Las reglas generales del chart están desactivadas para evitar avisos sin acción en este entorno. Alertmanager agrupa las alertas y cada una enlaza su runbook; no se configuraron notificaciones externas.

El SLI de disponibilidad HTTP no identifica errores GraphQL devueltos con HTTP 200. Para operación prolongada falta añadir un indicador semántico de esos errores y evaluar las alertas con un historial completo; no se atribuye a esta prueba una cobertura que no mide.